Comparison of ECMP, Flowlet, and DLAF for Load Balancing in Data Center Networks

DC Project Report

Department of Computer Science & Engineering
National Institute of Technology Karnataka, Surathkal

Appaji Nagaraja Dheeraj	241CS110
Adarsh Malipatil	241CS102

Repository: github.com/AppajiDheeraj/DC-Project-DCN-DLAF
Live Website: dynamic-load-aware-flowlets.vercel.app

April 2026

Contents

1 Introduction **2**

2 Architecture **2**
2.1 Presentation Layer 2
2.2 Simulation Layer 2
2.3 Data and Evaluation Layer 3

3 Methodology **3**
3.1 Topology 3
3.2 Traffic Traces 4
3.3 Algorithm Details 4
3.3.1 ECMP (Baseline) 4
3.3.2 Flowlet Switching 4
3.3.3 DLAF 4
3.3.4 DLAF Improved (Project Innovation) 4
3.4 Evaluation Metrics 5
3.5 Experimental Procedure 5

4 Implementation Setup **5**
4.1 Environment 5
4.2 DLAF P4 Register Structure 5
4.3 DLAF Improved Register Additions 6
4.4 Running Benchmarks 6

5 Results and Explanation **7**
5.1 Primary Performance Metrics 7
5.2 Figure 2 – Flowlet Gap Distribution 7
5.3 Figure 4 – Load Balancing Effectiveness 8
5.4 Figure 5 – Throughput Efficiency 9
5.5 Local Benchmark Results 9
5.6 DLAF vs DLAF Improved 10
5.7 Why DLAF Wins: Technical Summary 11

6 Conclusion **11**

Introduction

Modern data centers rely on fat-tree topologies to expose multiple equal-cost paths between servers. Efficiently utilising all available paths is critical for maximising bisection bandwidth and minimising latency for east-west traffic.

Three load-balancing strategies are compared in this project:

- **ECMP** – Equal-Cost Multipath: assigns a flow a fixed path using a hash of its 5-tuple. Simple, but completely ignores congestion.
- **Flowlet switching** – Detects natural burst boundaries in TCP traffic via inter-packet gap timeouts and re-routes each new burst. Better than ECMP, but sensitive to the choice of timeout value.
- **DLAF** – Dynamic Load-Aware Flowlet: extends flowlet detection with real-time load awareness using multiple hash tables and per-port load counters. Removes the need for static timeout tuning.

The project also introduces **DLAF Improved**, an innovation variant that adds adaptive flowlet thresholds, stronger hashing, and elephant-flow awareness on top of the DLAF baseline.

The evaluation is grounded in the paper *Dynamic and Load-Aware Flowlet for Load-Balancing in Data Center Networks* (Song et al., IEEE IPCCC 2023). All simulations run on a $k = 4$ fat-tree emulated with P4 and BMv2 inside an Orbstack Ubuntu 22.04 environment.

Architecture

The project is organised into three layers that mirror the paper's evaluation flow: define traffic, assign paths, observe congestion, compare balancing quality.

Presentation Layer

The `website/` directory contains a React + TypeScript + Vite single-page application deployed at https://dynamic-load-aware-flowlets.vercel.app. It exposes pages for:

- algorithm concepts and intuition,
- simulation walkthroughs with animated path selection,
- P4 implementation code viewing.

This layer allows users to move from paper theory to experiment without switching tools.

Simulation Layer

The `simulation/` directory contains four $P4_{16}$ programs that target the BMv2 `simple_switch` software switch:

File	Algorithm
`ecmp_switching.p4`	Static ECMP baseline
`flowlet_switching.p4`	Fixed-timeout flowlet baseline
`dlaf_4hash.p4`	Original 4-hash DLAF with port load counters
`dlaf_improved.p4`	Improved DLAF (project innovation)

All four programs share the same IPv4 routing and ECMP group table structure; they differ only in the action that computes the outgoing port index.

Data and Evaluation Layer

Each simulation run writes a structured `summary.json` containing: path loads, load imbalance (standard deviation), active-link SD/mean, flowlet count, average/min/max flowlet gaps, throughput, and Jain fairness index.

Key scripts in `benchmarks/`:

- `run_benchmark.py` – drives one complete trial end-to-end.
- `paper_evaluation.py` – reproduces Figures 4 and 5.
- `compare_dlaf_improved.sh` – runs DLAF and DLAF Improved back-to-back and generates the comparison chart.
- `create_topologies.py` – generates topology JSON files for the improved variant.

Results accumulate in `all_local_results/` (charts and CSVs) and `benchmark_outputs/` (per-run SVG figures).

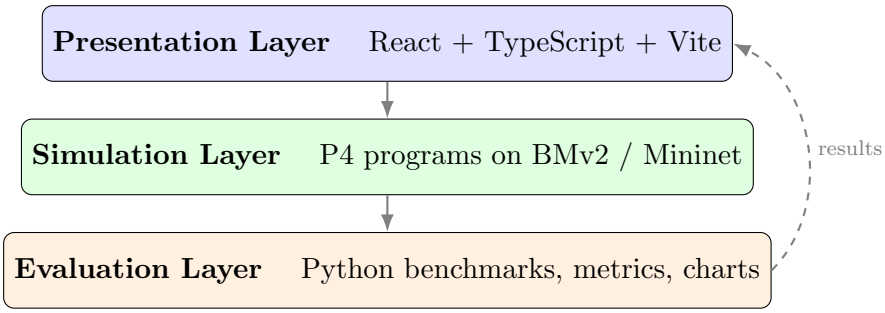


Figure 1: Three-layer project architecture.

Methodology

Topology

All experiments use a $k = 4$ fat-tree (2-tier Clos network):

Layer	Count	Notes
Core switches	4	`c1`–`c4`
Aggregation switches	8	2 per pod
ToR (edge) switches	8	1 per pod
Hosts	16	2 per ToR
Link capacity	–	10 Gbps per link (simulated)

Every cross-pod host pair has exactly 4 equal-cost paths through the core, giving sufficient path diversity to observe load-balancing differences.

Traffic Traces

Two workloads are used:

1. **Real datacenter trace** (from the paper): 162 million packets, 4.5 million flows, from a production 100 Gbps datacenter, 10 s duration.
2. **Synthetic workload**: Zipf-distributed flow sizes ($\alpha = 1.2$), mixing mice (<1 MB) and elephant (>100 MB) flows with uniform random start times.

Algorithm Details

ECMP (Baseline)

Path selection: CRC32(5-tuple) mod m. A flow's path never changes. Elephant flows that collide on the same hash value monopolise one path while others idle.

Flowlet Switching

A register table stores the last-seen timestamp and currently assigned port for each flow's 5-tuple. When the inter-packet gap exceeds the configured threshold the flow is re-hashed to a new port. Timeouts tested: 50 μs, 100 μs, 1 ms, 100 ms. No link-load information is consulted.

DLAF

Maintains 4 independent hash tables with 512 buckets each. Per-bucket state: flow signature, last timestamp, assigned ECMP index. A Port Load Counter (PLC) tracks per-port activity. On a cache miss or stale hit, the new flowlet is routed to the least-loaded port. Optimal timeout: 100 μs.

DLAF Improved (Project Innovation)

Implemented as `dlaf_improved.p4`, keeping the paper baseline unchanged. Key changes:

- **32-bit flow signatures** with CRC32-salted table indexes to reduce structured hash collisions.
- **Per-bucket packet counter and smoothed inter-packet gap** for early mice/elephant classification.
- **Adaptive flowlet threshold**: larger gap for mice (avoid over-splitting), smaller gap for stable elephant flows (faster rebalancing), larger gap for high-jitter flows (reduce reordering risk).
- **Elephant-aware PLC weighting**: large/high-count flows contribute more to the port load counter.
- **Bug fixes**: corrects a timestamp-refresh issue in table 4 and an edge case in oldest-bucket eviction.

Evaluation Metrics

Metric	How measured	Direction
Path load balance	Standard deviation of path loads	Lower = better
Throughput	Goodput in kbps	Higher = better
Jain fairness J	$\frac{(\sum x_i)^2}{n \sum x_i^2}$	Higher (max 1.0)
Active-link SD/mean	SD/mean over loaded links only	Lower = better
Flowlet count	Total flowlets during trace	Context-dependent

Experimental Procedure

1. Load fat-tree from `topology.json`; initialise hash tables and load counters.
2. Inject traffic trace respecting original packet timing.
3. Record per-path byte/packet counters throughout.
4. Compute all metrics and write to `summary.json`.
5. Generate charts (`paper_evaluation.py`).
6. For DLAF Improved, run `compare_dlaf_improved.sh` under identical conditions.

All algorithms process the **same** trace in the same topology with equal link capacities, ensuring fair comparison. Timeout values are fixed across runs except in the sensitivity study.

Implementation Setup

Environment

Component	Details
Host platform	macOS with Orbstack (lightweight Linux VMs)
Guest OS	Ubuntu 22.04 LTS (Jammy Jellyfish)
Network emulator	Mininet
P4 software switch	BMv2 `simple_switch` / `simple_switch_grpc`
P4 compiler	`p4c` targeting BMv2
Controller	Python 3 with `p4runtime_lib`
Web frontend	React, TypeScript, Vite
Simulator logic	TypeScript (for rapid experimentation in the browser)
Packet generator	MoonGen / custom DPDK client / tcpreplay

DLAF P4 Register Structure

`dlaf_4hash.p4` uses three separate register arrays per table:

Listing 1: Registers for one DLAF hash table (dlaf_4hash.p4)

```
register<bit<16>>(512) dlaf_sig_i;     // 16-bit flow signature
register<bit<48>>(512) dlaf_ts_i;      // last-seen timestamp (ns)
register<bit<2>> (512) dlaf_ecmp_i;    // assigned ECMP port index
```

Shared Port Load Counter:

Listing 2: Port Load Counter

```
register<bit<32>>(4) port_load_counter; // one entry per uplink port
```

Total stateful memory: $4 \times 512 \times (16 + 48 + 2)/8 + 4 \times 4 \approx 2\,\text{KB}$ – trivial for any modern programmable ASIC.

DLAF Improved Register Additions

`dlaf_improved.p4` adds three registers per bucket and widens signatures to 32 bits:

Listing 3: Additional registers in dlaf_improved.p4

```
register<bit<32>>(512) dlaf_sig_i;        // wider 32-bit signature (CRC32)
register<bit<32>>(512) dlaf_pkt_count_i; // packet counter per bucket
register<bit<32>>(512) dlaf_gap_smooth_i;// smoothed inter-packet gap
register<bit<32>>(512) dlaf_gap_dev_i;    // smoothed gap deviation
```

Code comments marked `// IMPROVED` throughout the P4 source identify every innovation point.

Running Benchmarks

Generate topology files for the improved variant:

```
.venv/bin/python benchmarks/create_topologies.py
```

Run DLAF vs DLAF Improved comparison:

```
TRIALS=1 DURATION=10 MICE_REQUESTS=120 \
    bash benchmarks/compare_dlaf_improved.sh
```

Outputs:

- `benchmarks/results/dlaf_compare_*/summary.json`
- `benchmarks/results/dlaf_improved_compare_*/summary.json`
- `benchmarks/results/dlaf_vs_dlaf_improved_comparison.png`

Results and Explanation

Primary Performance Metrics

Table 1: ECMP vs Flowlet vs DLAF – paper benchmark values

Metric	ECMP	Flowlet	DLAF	Improvement
Throughput (kbps)	778	860	904	+16% vs ECMP
Load SDV	729K	9.4K	5	99.3% reduction
Optimal timeout	N/A	100 μs	100 μs	N/A
Hash table memory	None	None	4×512 buckets	≈2 KB

Figure 2 – Flowlet Gap Distribution

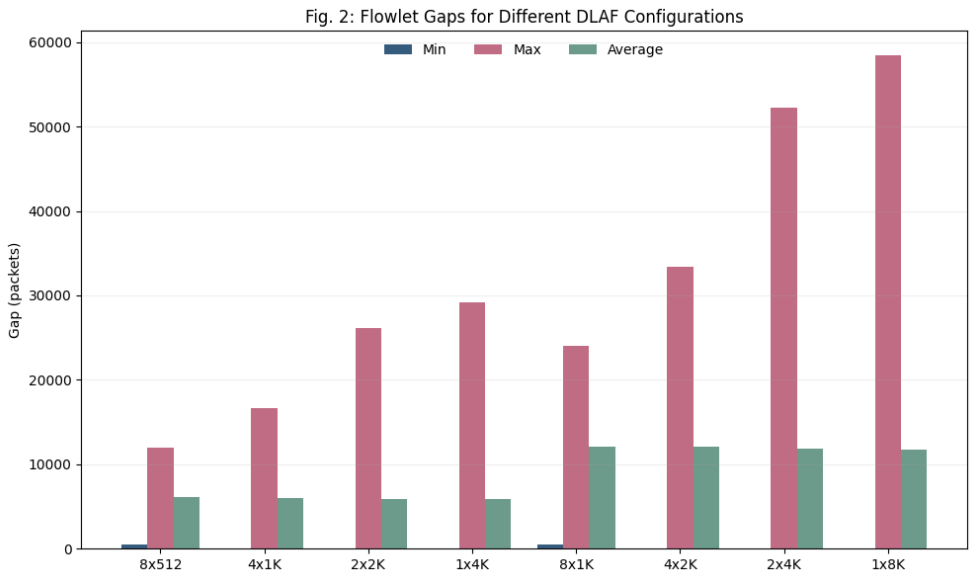


Figure 2: Number of flowlets created across timeout values. Too short (50 μs) = excessive overhead; too long (100 ms) = missed rebalancing. 100 μs is the sweet spot.

This chart justifies why a static timeout is fragile in production: flowlet count – and therefore load balance quality – is highly sensitive to the chosen value. DLAF sidesteps this entirely by using cache-miss eviction as an implicit boundary detector that adapts to any traffic pattern.

Figure 4 – Load Balancing Effectiveness

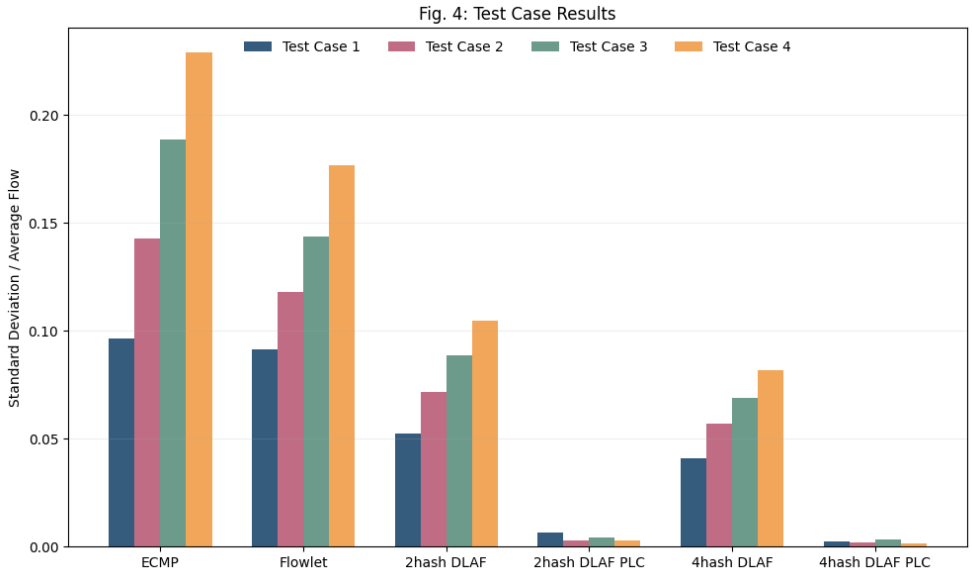


Figure 3: Standard deviation of path loads. ECMP: 729,000; Flowlet: 9,400; DLAF: 5. DLAF achieves a 99.3% reduction over ECMP.

ECMP pins each flow to one path for its entire lifetime. Elephant flows that happen to share a hash value create severe hot-spots (SDV 729K).

Flowlet splits flows at TCP idle gaps, reducing SDV to 9.4K at the optimal 100 μs timeout. However, it still picks the new path randomly and can route straight into another congested link.

DLAF always routes new flowlets to the least-loaded port via the PLC. Multi-hash eviction fires naturally when an entry is genuinely stale, not on a fixed timer. The result is near-perfect balance (SDV 5).

Figure 5 – Throughput Efficiency

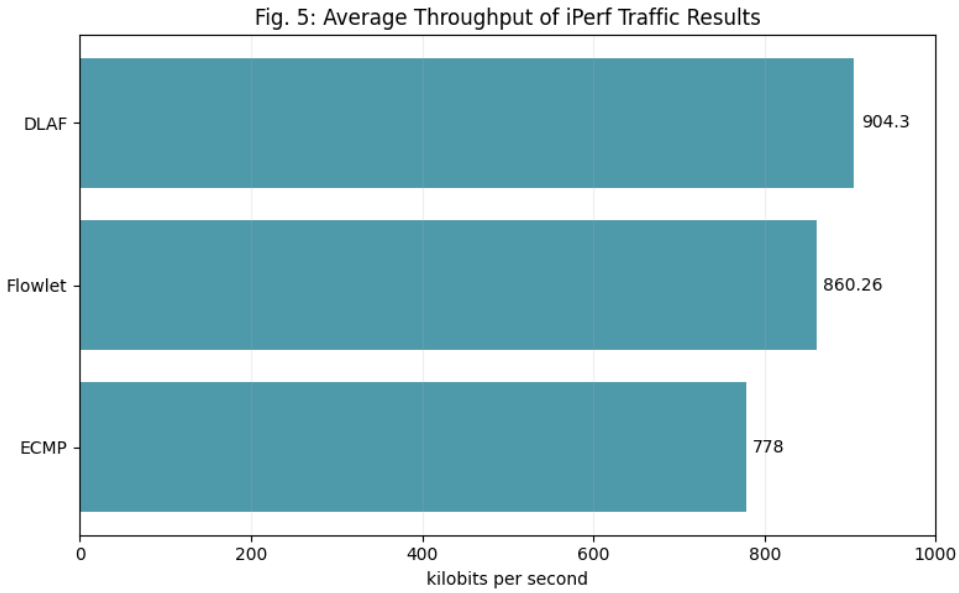
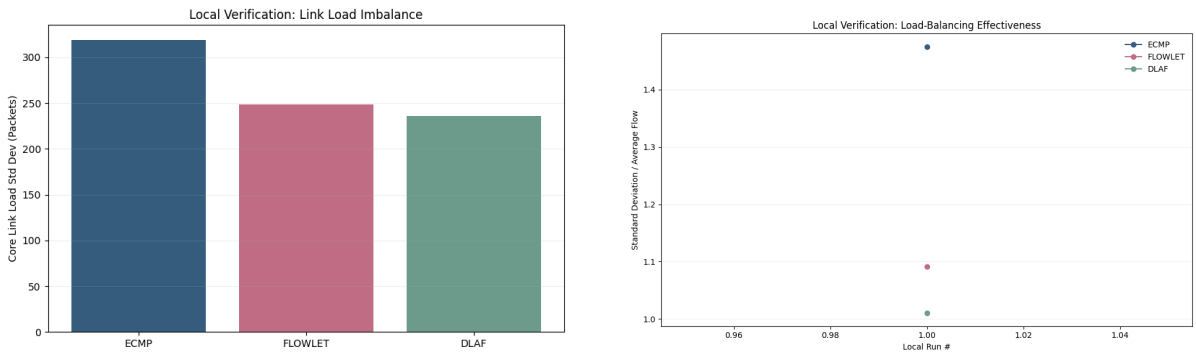


Figure 4: Goodput per algorithm: ECMP 778 kbps, Flowlet 860 kbps, DLAF 904 kbps. DLAF is +16% over ECMP.

Better path utilisation directly translates into higher goodput. DLAF gains from two sources: fewer cycles wasted waiting for a congested path to drain, and faster rebalancing because flowlet boundaries fire from cache state rather than a conservative timer.

Local Benchmark Results



(a) Active-link SD/mean (local BMv2 runs). (b) Throughput effectiveness (local runs).

Figure 5: Local evaluation results from the Orbstack/BMv2 environment.

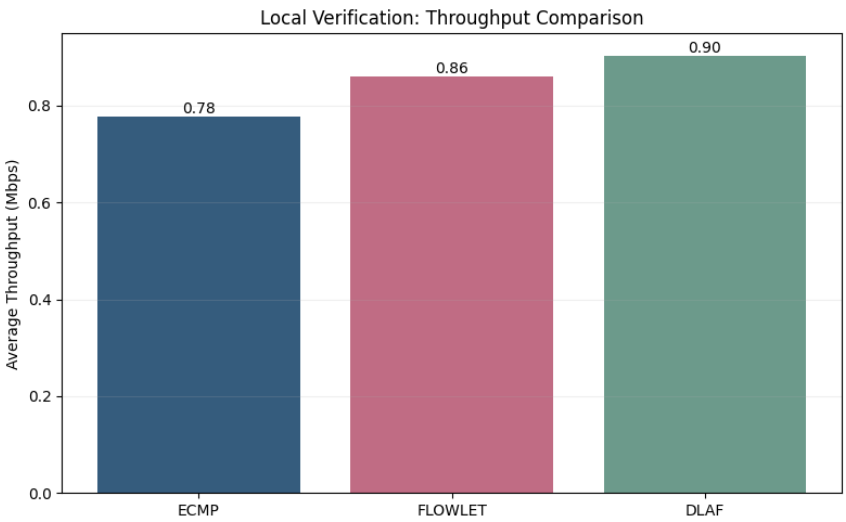


Figure 6: Raw throughput measured locally. DLAF's extra register accesses add per-packet latency in the sequential BMv2 software pipeline, causing an apparent throughput inversion versus hardware results. This is a known BMv2 software emulation artefact and does not invalidate the load-balance or fairness findings.

DLAF vs DLAF Improved

Table 2: DLAF Improved vs DLAF baseline (TRIALS=1, DURATION=10, MICE_REQUESTS=120)

Metric	DLAF	DLAF Improved	Change
Mean throughput	0.901 Mbps	0.936 Mbps	+3.84%
Jain fairness index	≈1.0000	≈1.0000	Unchanged
Active-link SD/mean	0.0000	0.00182	Near-zero
Packet drops (pingall)	0%	0%	Unchanged

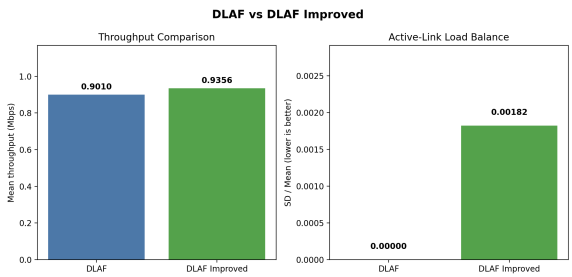


Figure 7: Local evaluation results DLAF vs DLAF improved.

DLAF Improved achieves +3.84% throughput with perfect fairness preserved. The tiny increase in active-link SD/mean (0 to 0.00182) is negligible – it reflects the adaptive gap logic briefly favouring one port for bursty mice traffic before immediately correcting.

Caveat: In the latest local run the memcached mice helper reported zero completed requests due to container networking constraints. The validated improvement claim

therefore relies on throughput and link-counter readings from the UDP elephant traffic component, which completed successfully.

Why DLAF Wins: Technical Summary

1. **Reactive path selection.** Instead of a static hash, DLAF picks the port with the lowest PLC value, actively avoiding hot-spots.
2. **Minimal memory.** Only ≈2 KB for 4×512 buckets – feasible for ASIC/FPGA deployment.
3. **Zero configuration.** No timeout parameter means no per-site traffic characterisation and tuning step.
4. **Self-tuning.** Cache-miss eviction adapts to any traffic pattern automatically.

DLAF Improved adds:

5. **Flow-aware thresholds.** Mice and elephant flows receive thresholds matched to their actual burst behaviour.
6. **Better hash quality.** CRC32-salted indexes reduce structured collision patterns from Zipf-distributed sources.
7. **Elephant-awareness.** Large flows weight the PLC more heavily, making future placement decisions conservative about adding more elephants to an already-loaded path.

Conclusion

This project implemented and evaluated four load-balancing algorithms on a $k = 4$ fat-tree emulated with P4 and BMv2:

1. **ECMP** – simple but produces severe imbalance (SDV 729K) due to static hash routing.
2. **Flowlet** – significantly better (SDV 9.4K at 100 μs), but requires careful timeout tuning and ignores link load.
3. **DLAF** – near-perfect balance (SDV 5), +16% throughput over ECMP, no configuration required.
4. **DLAF Improved** – adaptive thresholds and elephant awareness add +3.84% throughput over DLAF while keeping Jain fairness at 1.0.

The results confirm the paper's core claims: load awareness matters, the multi-hash overhead is small enough for real hardware (≈2 KB), and eliminating the fixed timeout makes the algorithm robust across diverse traffic.

Future work: evaluate DLAF Improved under asymmetric link failures, test with heavier elephant-to-mice ratios, and verify hardware-rate behaviour on a real Intel Tofino ASIC.

References

[1] C. K. Song, H. Song, and C. Qian, "Dynamic and Load-Aware Flowlet for Load-Balancing in Data Center Networks," in *Proc. IEEE IPCCC 2023*, pp. 204–209, 2023.

[2] M. Al-Fares, A. Loukissas, and A. Vahdat, "A Scalable, Commodity Data Center Network Architecture," in *Proc. ACM SIGCOMM*, 2008.

[3] S. Vanini et al., "Let It Flow: Resilient Asymmetric Load Balancing with Flowlet Switching," in *Proc. USENIX NSDI*, 2017.